“Wait, Cancel That”: Release Monotonicity and Cancellation Semantics for Streaming Voice-Agent Actions

Qalab Hassnain Agha
Co-Founder and Chief Technology Officer
Quickgen Technologies (SMC-Private) Limited
National Science and Technology Park, NUST H-12 Campus, Islamabad, Pakistan
qalab@quickgentech.com
ORCID: 0009-0006-6242-2761

1 November 2026

Abstract

Voice agents can begin tool work before a user finishes speaking, and users often revise mid-utterance: “room 305 ... actually, 503.” A correct final database does not show that the revision prevented the earlier effect, because a sent notification cannot be recalled. This is the output-commit problem of rollback recovery, met again at the turn boundary. We formalize it in a deterministic model with oracle-parsed intents that separates source intent from receipt, reversible writes from irreversible notifications, and prevention from compensation. We prove a release-monotonicity theorem: when the user cannot observe effects before the endpoint, version fencing guarantees that releasing a commit or notification later never adds a committed or notified version, in every run, for any gate time and under any latency distribution. Holding effects until the system’s own endpoint decision therefore adds no acknowledgement latency and is optimal among causal policies that add none. Only irreversible effects need holding: a release gate that commits eagerly but holds notifications until the endpoint weakly dominates eager execution on every measured outcome, delaying third-party notifications by at most the gap between tool readiness and the endpoint. It helps exactly when a

correction arrives in that gap. An exhaustive paired check over 1,764 scenarios, seven policies and endpoint decisions from 600 to 1,400 ms finds zero violations. The results concern execution semantics under oracle intent, not speech recognition or production systems.

Keywords: voice agents; cancellation; output commit; speculative execution; version fencing; compensation; turn-taking.

1. Introduction

Consider a user who says, "Send housekeeping to room 305 ... actually, 503." An agent that starts work after the first room number saves time but may dispatch to the wrong room. If it then corrects its database, the final state is right although a member of staff has already received a room-305 notification. "Cancelled" collapses different outcomes: the effect was prevented, the database was compensated, or an irreversible message was already sent.

Rollback recovery delays output to the outside world until it is known to be safe [11, 12], and external synchrony showed this can be free for the user [13]. Voice agents face the same problem, except that what is not yet final is the user's intent rather than data. Early execution is real: some agents execute tools while listening [5] or call them speculatively [9], and in two public implementations we probed, tool dispatch carries no intent version at all (Section 6.8). We ask when a voice agent may release an irreversible effect, and at what cost.

We isolate execution semantics from recognition: the model receives a perfect stream of structured intent events and processes no audio. RQ1 asks whether final database correctness distinguishes prevention from recovery. RQ2 asks when delaying release reduces superseded effects, and at what latency cost. RQ3 asks what version fencing guarantees when corrections are delivered with delay.

The contributions are: (1) an operational model separating source from received intent, reversible from irreversible effects, and prevention from compensation; (2) a release-monotonicity theorem with its consequences and its boundary: the exact condition under which delay helps, zero cost and causal optimality of an endpoint gate, weak dominance of a gate that holds only the irreversible effect, non-monotone stale database writes, and unfenceable corrections in transit; and (3) an exhaustive, auditable consistency check

of the proofs. Fencing, compensation and buffered output are not new, and the theorem is short; its value lies in locating the zero-cost release point for spoken intent and stating precisely what it protects.

2. Related Work

2.1 Voice agents, turn-taking and self-repair

Full-Duplex-Bench-v3 evaluates spoken systems on human audio with disfluencies and multi-step tool use [1]. Qwen-Audio-Agent runs background execution beside foreground conversation and separates speech interruption from task cancellation [2]; DuplexAgent-RSI handles cancellation, replacement and result delivery in a modular harness [3]. VAmoS Bench scores action traces and backend changes rather than spoken assertions [4]; our metrics add that a backend ending correct may still have emitted an irreversible superseded effect.

Spoken dialogue systems decide when the user's turn has ended from the audio received so far, under uncertainty [14]. Self-repairs typically follow the trouble source closely, within the same turn [15, 16]. These facts motivate separating corrections during the turn from changes of mind after it, and gating on the system's own endpoint decision.

2.2 Speculation, output commit and recovery

Incremental dialogue processing formalizes revocation and commitment of processing units [8]. Heartbeat Thinking executes tools while listening [5]; Speculative Actions predicts and validates future actions [6]; our model performs no prediction. Speculative Interaction Agents run early work and hold tools they classify as sensitive for explicit confirmation [9]. That is the closest precedent to our gate. We show which effects need holding (the irreversible ones), until when (the system's own endpoint decision, without asking the user), and that the hold costs no acknowledgement latency.

We transfer the output-commit principle [11, 12, 13] from data durability to intent finality and identify the endpoint decision as the point at which the delay is free. Three features have no counterpart in durability: intent can be revised, not only made stable; a correction can be in transit when an effect is released (Propo-

sition 3); and stale database writes are not monotone in delay. Our version check is a fencing token [17], a form of optimistic validation [18]; compensation follows Sagas [7].

3. Operational Model

3.1 Scope and intent stream

Each scenario has one mutable resource: a room assignment, initially empty. The initial instruction targets room 305, a revision targets 503, a second revision targets 701, and a cancellation specifies an empty assignment. A non-cancelling interruption leaves the instruction unchanged and does not advance the version.

Each substantive intent has an increasing integer version. A source event changes the intended target at virtual time s; the runtime receives it at r = s + d, with a fixed delivery delay d for events after the initial one, which is delivered at zero. W(t) is the latest source version and R(t) the highest version received by t. R is nondecreasing, which a runtime can guarantee under reordering by ignoring older versions. A delayed correction creates an interval in which W(t) exceeds R(t).

3.2 Preparation, commitment and notification

Preparation takes a duration P and has no external side effects. The policy then chooses when to attempt the database commit and the notification: their release times. A notification is irreversible once accepted. With fencing, a commit or notification succeeds only if its version equals R at that instant, and the check and state change are one atomic event. A real system must enforce this at the authoritative boundary, for example with a version predicate in the write and a transactional outbox whose dispatcher rechecks the version; an application-level check followed by an unprotected write is not covered.

3.3 Compensation

When a substantive update is received while the database is nonempty, compensation is scheduled after C milliseconds. It captures the version present at receipt and clears the assignment only if that version is still present, so a delayed recovery cannot delete a newer assignment. Compensation never installs a version or

touches the notification history, and it always succeeds here. A revision also schedules preparation of its replacement; a cancellation schedules none.

3.4 Event ordering

Time is integer virtual milliseconds. At equal timestamps, events execute in the order source updates, receipts, compensation, commits, notifications, so a correction received exactly at a release blocks it. The queue is drained completely; there are no timeouts, losses, duplicates or random scheduling, and overlap is unlimited.

4. Policies and Release-Timing Results

4.1 Policies

For an operation received at r with preparation duration P, the policies release effects at these times, where E is the end-of-turn gate:

```
Policy commit attempt notification attempt
Wait max(r, E) + P commit + N
Eager r + P commit + N
Guarded(g) max(r + P, E + g) commit + N
Release gate r + P max(r + P, E) + N
```

Guarded(g) prepares on receipt but holds the commit until E + g, with g in {0, 200, 500} ms. The release gate commits as soon as preparation completes but holds the irreversible notification until the gate. All six share fences and compensation; Guarded-no-fence removes both fences as a diagnostic ablation. The main grid fixes E at 1,000 ms, but every result holds for any gate time, including the system's own endpoint decision $\hat{E}$, which is internal state rather than an oracle (Section 6.2).

4.2 Freshness and convergence

Proposition 1. With fencing and atomic check-and-write, no commit or notification is stale relative to the latest received substantive intent at that instant.

Proof. A fenced event succeeds only if its version equals R at the atomic event, so an event whose version differs from R is rejected. Source updates not yet received do not change R, so the proposition says nothing about freshness relative to W.

Proposition 2. Under a finite intent stream, reliable delivery, finite preparation, successful compensation and fencing, the database eventually equals the final target.

Proof. After the last receipt no older operation can commit. A final set operation commits once prepared, and pending compensation can clear only older captured versions. After a final cancellation, older commits are blocked and any assignment is cleared by its scheduled compensation. Convergence says nothing about effects already emitted.

Proposition 3. Received-version fencing alone cannot guarantee freshness against corrections delivered after a release.

Proof. Consider two executions with the same received history before release time c. In one, no correction exists; in the other, a correction occurs at s < c but arrives after c. A deterministic runtime has identical information at c in both and makes the same decision, so if it releases in the first it releases an effect stale relative to source intent in the second.

4.3 Release monotonicity

Throughout this section the intent stream is exogenous: it does not depend on the policy. This holds when the user cannot observe effects released before the endpoint, as in half-duplex agents.

Lemma 1. Let operation v be received at r_v. If its fence passes at time t2, it passes at every t1 with r_v <= t1 <= t2.

Proof. R is nondecreasing and R(t) >= v for t >= r_v, so v <= R(t1) <= R(t2) = v. The argument holds for any fixed equal-time ordering, because R remains nondecreasing in event order.

Theorem 1 (release monotonicity). Let two fenced policies A and B face the same intent stream. (i) If B attempts every commit no earlier than A attempts the same commit, then every version B commits is committed by A, and B prevents every cancellation that A prevents. (ii) If B attempts every notification no earlier than A attempts the same notification, then every version B notifies is notified by A. Under the

respective condition, B therefore has no more superseded commits, or superseded notifications, than A.

Proof. Receipts are fixed by the intent stream, so R is the same function of time under both policies, and every received set operation is scheduled under both. For (ii), suppose B notifies v at time n_B, so the fence passes at n_B. A attempts the commit of v at some c_A >= r_v and its notification at n_A, with c_A <= n_A <= n_B. By Lemma 1 the fence passes at c_A and at n_A, so A commits and notifies v. Part (i) is the same argument applied to commit times. Compensation neither installs versions nor alters notifications, so it changes neither set. Superseded effects are the members of these sets other than the final version, and a cancellation is prevented exactly when the commit set is empty.

Scope of Theorem 1. The proof uses neither the single resource nor the fixed durations. It holds for any number of resources and dependent actions, provided each effect is fenced against the version of the instruction that authorized it and both policies attempt the same operations; revised partial hypotheses of a streaming recognizer are further version increments and are covered unchanged. Because the inclusion holds in every run, it holds almost surely under any distribution of preparation times and delivery delays that does not depend on the policy, so expected superseded counts are ordered too. Equal-time ordering affects only release times of measure zero in continuous time.

Non-atomic dispatch. External channels rarely support a conditional send. Suppose a dispatcher checks the version at time k and the effect becomes irreversible at k + w. Lemma 1 concerns only when the fence is evaluated, and every check follows the operation's receipt, so Theorem 1 holds with check times in place of release times. A correction arriving in (k, k + w] is no longer stopped: the race window weakens every policy's protection by the same w but preserves their ordering.

Corollary 1 (exact gain). With a single correction received at r, a fenced policy emits a superseded notification if and only if its initial notification attempt n0 satisfies n0 < r. Delaying that attempt from n0 to n0' therefore strictly helps exactly when n0 < r <= n0'. For Eager versus Guarded(0), this is P0 + N < r <= max(P0, E) + N: the correction must arrive after the tool is ready but before the gate. (Revision chains are covered by Theorem 1; this corollary is the single-correction case.)

Proof. The revised operation is final, so only the initial version can be superseded. If r <= n0, the fence blocks the initial notification (or its commit, which precedes it). If n0 < r, the commit and notification both precede the receipt and pass.

Corollary 2 (only the irreversible effect needs the gate). The release gate and Guarded(0) attempt every notification at the same time, so applying Theorem 1(ii) in both directions shows that they notify exactly the same versions in every scenario. The release gate commits the database earlier; its extra commits are reversible and recovered by compensation.

Proposition 4 (zero acknowledgement cost). Suppose the agent acknowledges an instruction only after its authoritative commit and only after its own endpoint decision $\hat{E}$, so the acknowledgement time is max(c_ final, $\hat{E}$). For any $\hat{E}$, a gate at $\hat{E}$ (Guarded with E = $\hat{E}$, or the release gate) has the same acknowledgement time and the same preparation work as Eager.

Proof. Eager commits at r + P. Guarded commits at max(r + P, $\hat{E}$), and max(max(r + P, $\hat{E}$), $\hat{E}$) = max(r + P, $\hat{E}$). The release gate commits at r + P. All three prepare on receipt.

The cost does not vanish; it moves to third parties. The release gate delivers a notification max(0, $\hat{E}$ - (r + P)) later than Eager, which matters only for latency-critical external actions, and those are excluded from the claim. With $\hat{E}$ = E, the acknowledgement time minus E is the stable residual latency of Section 6.4: P for Wait, max(P - E, g) for Guarded(g), and max(0, P - E) for Eager and the release gate. Commit times still differ (for P < E, Guarded(0) commits at E and Eager at P), but a user acknowledged after the endpoint cannot perceive it.

Theorem 2 (optimality of the endpoint gate). Call a fenced policy causal if each decision at time t depends only on intents received by t. Call it zero-added-latency if it prepares on receipt and acknowledges, for every finite intent stream, at max(r_final + P_final, $\hat{E}$): the earliest time at which any policy that prepares on receipt can acknowledge. Among causal zero-added-latency policies whose notification follows its commit by N, the gate at $\hat{E}$ notifies a subset of the versions notified by any other, in every scenario.

Proof. Fix a scenario, an operation v with ready time u = r_v + P_v, and b = max(u, $\hat{E}$). If a newer version is received by b, the gate never commits v. Otherwise, consider the stream that agrees with this one up to b and contains no later intents; there v is final, so a zero-added-latency policy must commit v no later than b. The endpoint decision $\hat{E}$ <= b is itself made from input received by then. A causal policy cannot distinguish the two streams before b, so it attempts v no later than b in the original scenario as well, and its notification no later than b + N, when the gate notifies. Lemma 1 then gives the inclusion as in Theorem 1.

What is not monotone. Delay can add commits that are stale relative to source intent. A later commit

can land inside an interval [s, s + d) in which a correction exists but has not yet been received, while an earlier commit would have preceded s. Theorem 1 still bounds the notifications of such commits; the stale database state itself is reversible and is recovered by compensation. This explains why, in Section 6.6, Wait shows more world-stale commits than Guarded(0) even though it never commits earlier. It also means that a commit gate does not dominate Eager on every outcome. The release gate does: its commits coincide with Eager's, so every commit-based outcome is identical, and by Theorem 1(ii) its notifications are a subset of Eager's.

5. Experimental Design

5.1 Fixed scenario grid

Table 1. Factors of the scenario grid.

Factor	Values
Intent pattern	Stable; non-cancelling interruption; revision; cancellation; two successive revisions
First change time	150, 450, 850, 1050, 1350, 1800 ms; absent for stable cases
Initial preparation P0	100, 500, 1200 ms
Revised preparation P1	100, 500, 1200 ms for revision and chain cases
Delivery delay d	0, 100, 350 ms for subsequent events
Notification delay N	0, 200 ms
Compensation delay C	0, 250 ms
End-of-turn gate E	1000 ms (600 to 1400 ms in the endpoint sweep)

In a chain, the second change occurs 200 ms after the first and uses P0. The grid has 36 stable, 216 interruption, 648 revision, 216 cancellation and 648 chain scenarios (1,764); seven policies yield 12,348 runs. The grid checks the per-scenario theorems and illustrates magnitudes; its percentages depend on the chosen values and do not estimate user behaviour. We stratify by whether the first correction falls during the turn (150, 450, 850 ms) or after it. The enumeration is exhaustive, so confidence intervals would not be meaningful.

5.2 Outcome definitions

Final-state correctness: the database at quiescence equals the last source target. **Known-stale** and **world-stale commit incidence:** some commit's version differs from R, or from W, at commitment. **Superseded commit or notification incidence:** among revision, cancellation and chain scenarios, some committed or notified version differs from the final one; such an effect may have been legitimate when emitted. **Prevention:** no database commit (respectively, no notification) occurred in a cancellation scenario. **Recovery:** the final database is empty. **Residual settlement latency:** the time at which the database becomes continuously equal to the final target, minus max(E, last substantive source time), clipped at zero; it excludes audio generation and notification delivery.

5.3 Reproducibility and validation

The standard-library Python implementation makes no random draws. It emits scenarios, per-run outcomes, event traces, aggregate, stratified and paired results, and a manifest of SHA-256 hashes. Thirty tests cover analytic latency, cancellation before and after commit, equal-time ordering, delayed delivery, unfenced stale writes, notification suppression and permanence, compensation races, the release gate, and full-grid convergence and monotonicity. A separate audit recomputes the metrics from saved traces. The simulator, tests and analysis scripts were developed with AI coding assistance, as declared at the end of the paper.

6. Results

6.1 Paired verification of release monotonicity

For each ordered pair of policies in Theorem 1, we compare the sets of committed and notified versions scenario by scenario (Table 2). The simulator is an independent implementation of the model, so these comparisons check the proofs; they cannot check the model's assumptions.

Table 2. Paired per-scenario check of Theorem 1.

Earlier release	Later release	Notified-set violations	Commit-set violations	Prevention lost	Fewer superseded notifications	World-stale commit added
eager	guarded_0	0	0	0	398	48
guarded_0	guarded_200	0	0	0	152	48
guarded_200	guarded_500	0	0	0	228	72
guarded_0	wait	0	0	0	328	72
eager	release_gate	0	0	0	398	0

Each row: 1,764 paired scenarios. Violation and loss columns count scenarios that contradict Theorem 1; the last two columns count scenarios.

No scenario contradicts Theorem 1, and Guarded(0) has strictly fewer superseded notifications than Eager in 398 scenarios. The release gate and Guarded(0) notify identical version sets in 1,764 of 1,764 scenarios (Corollary 2), and Corollary 1 reproduces 5,184 of 5,184 single-change outcomes across the six fenced policies. The world-stale column is the exception predicted in Section 4.3: Guarded(0) adds world-stale commits relative to Eager in 48 scenarios, so it does not dominate Eager on every outcome, while the release gate adds none. Guarded(500) and Wait are not ordered, and each has strictly fewer superseded notifications in some scenarios (98 and 146), so their aggregate ranking reflects grid weighting only.

6.2 Any endpoint decision

We replaced E with endpoint decisions of 600 (early), 800, 1,000, 1,200 and 1,400 ms (late) and compared Eager with each successively later gate over all 1,764 scenarios. The five steps produce 0 notified-set or commit-set violations in total and strictly reduce superseded notifications in 162, 102, 146, 152 and 144 scenarios respectively. Every gate, even one 400 ms before the nominal end of the turn, is never worse than Eager on notifications, commits or prevention; as Section 4.3 explains, world-stale database commits are the exception.

6.3 Corrections during versus after the turn

Table 3. Superseded notifications for corrections during and after the original turn.

Policy	Correction during turn	Correction after turn
wait	3.17%	44.71%
eager	49.74%	89.42%
guarded_0	5.56%	82.01%
guarded_200	0.79%	68.25%
guarded_500	0.00%	41.27%
release_gate	5.56%	82.01%

Superseded-notification incidence. Denominators: 756 changed scenarios with first change before E and 756 at or after E, per policy.

These rates depend on the chosen change times and durations and illustrate Corollary 1 on this grid; they do not estimate deployed behaviour. For corrections made during the original turn, the gate at the endpoint reduces superseded notifications from 49.74% (Eager) to 5.56% (Guarded(0) and the release gate) at the same acknowledgement latency. The residue arises where a receipt falls after the gate (Corollary 1), through delivery delay or a chain's second revision landing after the turn. For changes of mind after the turn, a zero-cost gate barely helps (89.42% to 82.01%). Protecting against them requires holding beyond the endpoint, as Guarded(500) does (41.27%), at a latency cost: the genuine trade-off.

6.4 Aggregate outcomes

Across all 1,764 scenarios, every fenced configuration reaches 100% final-state correctness with no known-stale commit (Propositions 1 and 2). On the 1,512 changed scenarios, superseded-notification incidence ranges from 20.63% for Guarded(500) to 69.58% for Eager, with 43.78% for Guarded(0) and the release gate. Stable residual latency is 66.7 ms for Eager, Guarded(0) and the release gate, although mean commit times are about 600 and 1,066.7 ms; it is 200 and 500 ms for Guarded(200) and Guarded(500) and 600.0 ms for Wait. **Table 4.** Aggregate outcomes over the full grid.

Policy	Final correct	Known-stale commit	World-stale commit	Superseded notification*	Stable latency (ms)
wait	100.00%	0.00%	10.88%	23.94%	600.0
eager	100.00%	0.00%	12.24%	69.58%	66.7
guarded_0	100.00%	0.00%	9.52%	43.78%	66.7
guarded_200	100.00%	0.00%	8.84%	34.52%	200.0
guarded_500	100.00%	0.00%	7.48%	20.63%	500.0
release_gate	100.00%	0.00%	12.24%	43.78%	66.7
guarded_no_fence	79.37%	55.10%	62.13%	100.00%	66.7

Each policy: 1,764 runs. *Changed-scenario denominator: 1,512 per policy. Stable latency: 36 stable scenarios per policy.

6.5 Prevention differs from recovery

Table 5. Prevention and recovery in cancellation scenarios.

Policy	Cancellation scenarios	No database commit	No notification emitted	Final empty database
wait	216	75.93%	78.70%	100.00%
eager	216	27.78%	32.41%	100.00%
guarded_0	216	50.00%	57.41%	100.00%
guarded_200	216	61.11%	66.67%	100.00%
guarded_500	216	77.78%	80.56%	100.00%
release_gate	216	27.78%	57.41%	100.00%
guarded_no_fence	216	0.00%	0.00%	50.00%

All fenced policies recover the empty state, yet prevention differs widely, so a metric restricted to final database correctness would erase the distinction. The release gate separates the two effect classes: it prevents the database write as rarely as Eager (27.78%) but prevents the notification exactly as often as Guarded(0) (57.41%), as Corollary 2 implies. An emitted notification survives compensation even when the database later becomes empty.

6.6 Delayed corrections

Table 6. World-stale commit incidence by delivery delay.

Policy	d = 0 ms	d = 100 ms	d = 350 ms
wait	0.00%	14.29%	23.81%
eager	0.00%	19.05%	23.81%
guarded_0	0.00%	9.52%	23.81%
guarded_200	0.00%	7.14%	23.81%
guarded_500	0.00%	4.76%	21.43%
release_gate	0.00%	19.05%	23.81%

World-stale commit incidence among changed scenarios: 504 per policy and delivery-delay setting.

With zero delay, W = R and no fenced policy commits world-stale (Proposition 1); positive delay creates exposure that gating reduces but cannot remove (Proposition 3). Wait never commits earlier than Guarded(0) yet shows more world-stale commits at d = 100 ms, because its later commits fall inside the delivery interval of corrections at 1,050 and 1,350 ms (Section 4.3).

6.7 Fencing ablation and counterexample

Guarded-no-fence reaches 79.37% final correctness (91.36% for revisions, 50.00% for cancellations, 69.14% for chains). With P0 = 1,200 ms, a revision at 150 ms and P1 = 100 ms, the original becomes ready after the revision has committed: fenced, it is rejected; unfenced, it overwrites room 503 with 305. The ablation shows why late completions must be fenced, not superiority over published systems.

6.8 Authoritative-boundary testbed

A SQLite testbed with controlled coroutine barriers injects a correction before tool entry, after an application-level check, after the commit, or after the notification (eight cases). An application check followed by an unguarded write accepts an obsolete write; a version predicate at the write rejects it. Compensation restores the database without restoring prevention, and an accepted notification survives an empty final database. We also extracted the tool-dispatch functions of two pinned public implementations, the Speculative Actions retail tools [19] and an OpenAI Realtime agent [20], and retracted authorization before tool entry. Neither accepts an intent version, so both produced the effect; a local version check suppressed it. The fence is thus absent at those interfaces and implementable at a real write boundary; these are not failure-rate estimates or claims about the full systems.

7. Discussion

The runtime rule is: carry intent versions to the authoritative commit and dispatch boundaries, and never release an irreversible effect before the system's own endpoint decision. By Proposition 4 and Theorem 2 this costs nothing a user can perceive; reversible work can proceed early (Corollary 2). Beyond the endpoint, holding longer trades latency for protection against later changes of mind, a single parameter with a proven floor whose value needs domain costs we do not estimate. Cancellation acknowledgements should also distinguish "prevented," "compensated," "notification already emitted" and "outcome unknown."

The practical gain depends on how often a correction arrives after a tool is ready but before the endpoint (Corollary 1). If real tools usually outlast the remaining utterance, that window is empty and the gate coincides with Eager. Measuring it needs tool-latency logs and timed self-repair data, not more simulation. A reliable execution layer also cannot fix a wrong intent stream: misclassified interruptions, missed corrections and misattributed speakers are likely dominant real failures.

8. Limitations

Assumptions of the theorems. The intent stream is exogenous. This is plausible for half-duplex agents but not for the full-duplex agents of [1, 2, 3, 5], which can act or backchannel mid-turn. There, early feedback such as "sending to 305" may draw out a correction sooner, and an eager policy with feedback could outperform the gate; that case is open. Acknowledgement waits for the authoritative commit. Third parties receive held notifications up to $\hat{E}$ - (r + P) later, so latency-critical external actions are excluded. The fence is evaluated atomically, or with a fixed race window for external channels (Section 4.3). Preparation has no external side effects; holds, reservations and billing are effects and must be gated too.

Construct validity. The experiments use one assignment and an immutable log (Theorem 1 does not need this); compensation always succeeds, though a field returning to empty need not restore business reality. Durations are chosen, not fitted. Intent parsing is perfect, with no ASR system, language model or participant. Crashes, duplicates and remote write uncertainty are not modeled.

Real speech. The 498 public Full-Duplex-Bench v1.5 metadata records [10] cannot supply cancellation ground truth: their labels describe overlap, not task intent. A follow-up needs consented speech with inde-

pendently annotated corrections and measured tool, notification and endpoint timings.

9. Conclusion

Final database correctness cannot characterize voice-agent cancellation, because irreversible effects survive compensation. When the user cannot observe effects before the endpoint, fencing makes later release monotone in every run. A gate at the system's own endpoint decision is therefore free in acknowledgement latency and optimal among causal policies that add none, and a gate holding only irreversible effects weakly dominates eager execution at a bounded delay to third parties. Corrections in transit remain unfenceable, and stale database writes are not monotone in delay. The gain in deployed systems depends on how often tools finish before users stop speaking.

Declaration of Competing Interest

The author declares no competing interests. The work was carried out independently of Quickgen Technologies (SMC-Private) Limited, which did not fund it, and no company data, recordings, customer records or products were used.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial or not-for-profit sectors, and it was not funded by Quickgen Technologies.

CRediT Authorship Contribution Statement

Qalab Hassnain Agha: Conceptualization, Methodology, Software, Formal analysis, Investigation, Validation, Writing: original draft, Writing: review and editing.

Data Availability

The simulator, scenario grid, event traces, paired-check results, audit program, tests and manuscript sources are openly available at https://github.com/qalab-e-hassnain/cancelbench under the MIT license. Public Full-Duplex-Bench v1.5 metadata was processed under its terms and is not redistributed. No new human-subject data were collected, and all room identifiers and events are fictional.

Declaration of Generative AI and AI-Assisted Technologies

During the preparation of this work the author used Claude Code (Anthropic), an AI coding assistant, to help implement the simulator and tests, draft and revise manuscript text, and draft proofs and literature notes. All numerical results come from executing the released code; the assistant generated no scores, participants or measurements. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

References

[1] Guan-Ting Lin, Chen Chen, Zhehuai Chen, and Hung-yi Lee. Full-Duplex-Bench-v3: Benchmarking Tool Use for Full-Duplex Voice Agents Under Real-World Disfluency. arXiv:2604.04847, 2026. https://arxiv.org/abs/2604.04847

[2] Chong Deng, Yunjie Ji, Yuxiang Kong, Xiangang Li, Xu Li, Binbin Zhang, Haina Zhu, and Jianheng Zhuo. Qwen-Audio-Agent Technical Report. arXiv:2609.25195, 2026. https://arxiv.org/abs/2609.25195

[3] Yingda Shen, Yuxiang Wang, Kunyu Feng, Qinke Ni, Jiaqi Li, Minghao Hsu, Junan Zhang, Dekun Chen, Yutong Bian, and Zhizheng Wu. DuplexAgent-RSI: Recursive Harness Improvement for Full-Duplex Voice Agent Collaboration. arXiv:2610.11299, 2026. https://arxiv.org/abs/2610.11299

[4] Joshua Meyer, Sahar Shayegan, Ritiz Tambi, Ali Khan, Sun Kim, Victor Shih, Mehdi Jamei, and Andi Partovi. VAmoS Bench: Voice Agent Simulation Bench. arXiv:2607.27453, 2026. https://arxiv.org/abs/2607.27453

[5] Zhenyu Zhang, Kewei Fu, Yongqi Tong, Chenzhi Zheng, Wei Wu, Jiang-Ming Yang, and Jianshe Li. Heartbeat Thinking: Speculative Reasoning and Tool Execution While Listening in Realtime Voice Agents. arXiv:2610.26233, 2026. https://arxiv.org/abs/2610.26233

[6] Naimeng Ye, Arnav Ahuja, Georgios Liargkovas, Yunan Lu, Kostis Kaffes, and Tianyi Peng. Speculative Actions: A Lossless Framework for Faster Agentic Systems. arXiv:2510.04371, version 2, 2026. https://arxiv.org/abs/2510.04371v2

[7] Hector Garcia-Molina and Kenneth Salem. Sagas. In Proceedings of ACM SIGMOD, 1987. Author technical-report version: https://www.cs.princeton.edu/techreports/1987/070.pdf

[8] David Schlangen and Gabriel Skantze. A General, Abstract Model of Incremental Dialogue Processing. Dialogue & Discourse 2(1), 83-111, 2011. https://clp.ling.uni-potsdam.de/publications/Schlangen-2011.pdf

[9] Coleman Hooper, Minwoo Kang, Suhong Moon, Nicholas Lee, Eric Wen, John Wawrzynek, Michael W. Mahoney, Yakun Sophia Shao, Amir Gholami, and Kurt Keutzer. Speculative Interaction Agents: Building Real-Time Agents with Asynchronous I/O and Speculative Tool Calling. arXiv:2605.13360, 2026. https://arxiv.org/abs/2605.13360

[10] Guan-Ting Lin, Jiachen Lian, Tingle Li, Qirui Wang, Gopala Anumanchipalli, Alexander H. Liu, and Hung-yi Lee. Full-Duplex-Bench: A Benchmark to Evaluate Full-Duplex Spoken Dialogue Models on Turn-Taking Capabilities. v1.5 dataset release, 2025. https://github.com/DanielLin94144/Full-Duplex-Bench/tree/main/v1_v1.5

[11] E. N. Elnozahy, Lorenzo Alvisi, Yi-Min Wang, and David B. Johnson. A Survey of Rollback-Recovery Protocols in Message-Passing Systems. ACM Computing Surveys 34(3), 375-408, 2002.

[12] Robert E. Strom and Shaula Yemini. Optimistic Recovery in Distributed Systems. ACM Transactions on Computer Systems 3(3), 204-226, 1985.

[13] Edmund B. Nightingale, Kaushik Veeraraghavan, Peter M. Chen, and Jason Flinn. Rethink the Sync. In Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2006.

[14] Gabriel Skantze. Turn-Taking in Conversational Systems and Human-Robot Interaction: A Review.

Computer Speech & Language 67, 101178, 2021.

[15] Willem J. M. Levelt. Monitoring and Self-Repair in Speech. Cognition 14(1), 41-104, 1983.

[16] Elizabeth E. Shriberg. Preliminaries to a Theory of Speech Disfluencies. PhD thesis, University of California, Berkeley, 1994.

[17] Mike Burrows. The Chubby Lock Service for Loosely-Coupled Distributed Systems. In Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2006.

[18] H. T. Kung and John T. Robinson. On Optimistic Methods for Concurrency Control. ACM Transactions on Database Systems 6(2), 213-226, 1981.

[19] Speculative Actions reference implementation, retail tools, commit dc938b9. https://github.com/naimengye/speculative-action

[20] Veris AI. riley-agent, openai-realtime-mini variant, commit f54b220. https://github.com/veris-ai/riley-agent

Appendix A. Reproduction

Run from the repository root using Python 3.9 or newer:

```
PYTHONPATH=src python3 -m unittest discover -s tests -v
PYTHONPATH=src python3 -m cancelbench.run --output results
PYTHONPATH=src python3 -m cancelbench.audit --results results
python3 scripts/build_paper.py
```

The run writes `monotonicity.json` with the paired checks, endpoint sweep, closed-form check and turn strata reported in Section 6. The build script inserts measured tables into the manuscript template and produces Markdown, HTML and LaTeX versions. Repeat execution produces byte-identical experiment artifacts on the same interpreter; the manifest records the interpreter version.

Appendix B. Minimal Execution Pseudocode

```
on substantive receipt(version, target):
```

```
    latest_received_version = max(latest_received_version, version)
    if database is nonempty:
        schedule compare-and-clear(database.version) after C
    if target is not cancellation:
        schedule prepared operation according to policy

on scheduled commit(operation):
    atomically:
        reject unless operation.version == latest_received_version
        database = (operation.version, operation.target)
    schedule notification at policy release time (commit + N, or max(commit, E) + N)

on notification(operation):
    atomically:
        reject unless operation.version == latest_received_version
        append operation to immutable notification history
```

Guarded-no-fence omits the two version checks. Obsolete preparations may still finish in fenced configurations, but their releases are rejected: cancellation invalidates authorization for pending effects rather than preempting the computation.